Compte-rendu technique

Lecture d'un bouton-poussoir et commande d'une LED externe par GPIO — microcontrôleur STM32F407VGTx (carte STM32F407G-DISC1)

Réalisé par :

Mohamed Amine Dhaoui • Ahmed Hafi • Ayetallah Cherif • Essra Gamoudi

1. En-tête et contexte

Matériel utilisé (identifié sur les photos) :

- Carte STM32F407G-DISC1 (réf. MB997E), microcontrôleur STM32F407VGTx (Cortex-M4, boîtier LQFP100), programmeur/débogueur ST-LINK/V2-A intégré
- Câble USB / mini-USB (alimentation et programmation) relié à un PC portable
- Platine d'essai (breadboard), LED rouge, bouton-poussoir à capuchon vert, résistances, fils de liaison

Objectif principal : réaliser une chaîne simple d'acquisition et de commande sur le microcontrôleur **STM32F407VGTx** : **lire l'état d'un bouton-poussoir** sur une entrée GPIO et **commander en conséquence une LED externe** câblée sur une sortie GPIO. Un test préliminaire de clignotement valide au préalable la sortie et la chaîne de programmation.

Remarque : le code source et la liste exacte des broches n'ayant pas été fournis, les extraits ci-dessous sont des codes de référence (bibliothèque HAL). Les lignes marquées « À ADAPTER » doivent être remplacées par les ports et broches réellement câblés.

2. Déroulement étape par étape

Étape 1 : Présentation et alimentation de la carte

La carte est alimentée et programmée via son port mini-USB ST-LINK ; le voyant rouge « PWR » allumé confirme l'alimentation. Les deux barrettes de broches latérales exposent les GPIO, dont les références (PA, PB, PC, PD, PE…) sont sérigraphiées sur le circuit, ce qui facilite le repérage.



Figure 1 — Carte STM32F407G-DISC1 connectée au PC : microcontrôleur central, ST-LINK (à droite), bouton bleu (RESET), bouton noir (USER), barrettes de GPIO.

Étape 2 : Câblage du montage sur breadboard

Le câblage relie les GPIO de la carte à la platine d'essai :

- **Sortie GPIO → résistance → LED → GND** : la résistance limite le courant (typiquement 220 à 330 Ω). Avec une sortie 3,3 V et une LED rouge (Vf ≈ 2 V), I = (3,3 − 2) / 330 ≈ 4 mA, sans danger pour la broche.
- **Bouton-poussoir** : une borne est reliée à l'entrée GPIO, l'autre à une référence d'alimentation (3V ou GND) ; une résistance de rappel (pull-up/pull-down) fixe le niveau logique au repos et évite une entrée flottante.
- Une **masse commune** (GND) entre la carte et la platine est indispensable.

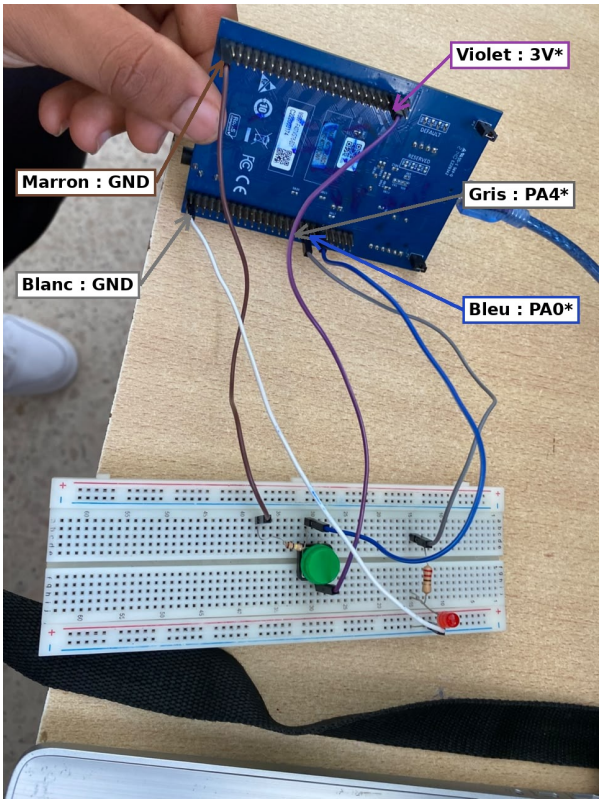


*Figure 2 — Face arrière de la carte et fils de liaison vers la breadboard, avec identification des broches : fil gris = sortie LED, fil bleu = entrée bouton-poussoir, fil violet = alimentation du bouton, fils marron et blanc = masse. * Broche à vérifier d'après le code.*

Étape 3 : Configuration des GPIO

Les ports GPIO sont sur le bus AHB1 : leur horloge doit être activée avant toute utilisation. La broche de la LED est configurée en **sortie push-pull**, celle du bouton en **entrée**, avec rappel interne si le montage n'en comporte pas d'externe.

```
/* ---- Définition des broches (À ADAPTER au câblage réel) ---- */
#define LED_PORT  GPIOA          // À ADAPTER (PA4)
#define LED_PIN   GPIO_PIN_4     // À ADAPTER (PA4)
#define BTN_PORT  GPIOA          // À ADAPTER
#define BTN_PIN   GPIO_PIN_0     // À ADAPTER (PA0)

static void GPIO_Init_Custom(void)
{
    GPIO_InitTypeDef gpio = {0};

    /* 1. Activation des horloges des ports (bus AHB1) */
    __HAL_RCC_GPIOA_CLK_ENABLE();

    /* 2. LED : sortie push-pull, basse vitesse */
    gpio.Pin   = LED_PIN;
    gpio.Mode  = GPIO_MODE_OUTPUT_PP;
    gpio.Pull  = GPIO_NOPULL;
    gpio.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(LED_PORT, &gpio);

    /* 3. Bouton : entrée numérique avec rappel */
    gpio.Pin  = BTN_PIN;
    gpio.Mode = GPIO_MODE_INPUT;
    gpio.Pull = GPIO_PULLDOWN;   // GPIO_PULLUP si bouton relié à GND
    HAL_GPIO_Init(BTN_PORT, &gpio);
}
```

Étape 4 : Programme 1 — Clignotement de la LED

La LED est basculée à intervalle régulier avec `HAL_GPIO_TogglePin` et une temporisation `HAL_Delay`, fondée sur le timer système SysTick (base de temps de 1 ms).

```
int main(void)
{
    HAL_Init();               // Initialisation HAL + SysTick (1 ms)
    SystemClock_Config();     // Horloges (générée par CubeMX)
    GPIO_Init_Custom();

    while (1)
    {
        HAL_GPIO_TogglePin(LED_PORT, LED_PIN);  // Inversion de l'état
        HAL_Delay(500);   // 500 ms ON / 500 ms OFF -> période 1 s
    }
}
```

Étape 5 : Programme 2 — Commande de la LED par bouton-poussoir

La broche du bouton est lue par scrutation (*polling*). La LED suit l'état du bouton, avec un anti-rebond logiciel simple pour filtrer les rebonds mécaniques.

```
while (1)
{
    if (HAL_GPIO_ReadPin(BTN_PORT, BTN_PIN) == GPIO_PIN_SET)   // Appui ?
    {
        HAL_Delay(20);                                         // Anti-rebond
        if (HAL_GPIO_ReadPin(BTN_PORT, BTN_PIN) == GPIO_PIN_SET)
            HAL_GPIO_WritePin(LED_PORT, LED_PIN, GPIO_PIN_SET);   // LED ON
    }
    else
    {
        HAL_GPIO_WritePin(LED_PORT, LED_PIN, GPIO_PIN_RESET);     // LED OFF
    }
}
```

Variante : inverser l'état de la LED à chaque appui (bascule), avec détection de front pour n'agir qu'une fois par pression.

Étape 6 : Compilation, téléversement et exécution

Le projet est créé pour la cible **STM32F407VGTx** (STM32CubeIDE), compilé, puis téléversé dans la Flash du microcontrôleur via le ST-LINK intégré (liaison USB, sans programmateur externe). Le programme démarre après le reset.

3. Présentation des résultats

Résultat obtenu : le montage répond conformément à la consigne : la LED externe est commandée par le bouton-poussoir, ce qui valide le câblage, la configuration des GPIO (entrée et sortie) et la chaîne de programmation sur le STM32F407VGTx.

Analyse de la vidéo de démonstration : la séquence (≈ 6 s) montre le montage de la Figure 2 en fonctionnement : la carte est alimentée par USB et reliée à la breadboard par les fils de liaison. Au départ (t ≈ 0 s), aucun appui n'est exercé et la LED rouge reste faible/éteinte. Dès que le doigt appuie sur le bouton vert (t ≈ 2 s), la LED rouge s'allume nettement et reste allumée tant que l'appui est maintenu (t ≈ 4 s). Le changement d'état est immédiat, sans scintillement perceptible, ce qui confirme que l'entrée GPIO détecte correctement l'appui et que la sortie pilote bien la LED. *(Seul un appui prolongé est visible : le comportement exact au relâchement, extinction ou bascule, dépend du programme retenu : version « suiveur » de l'étape 5 ou variante bascule.)*

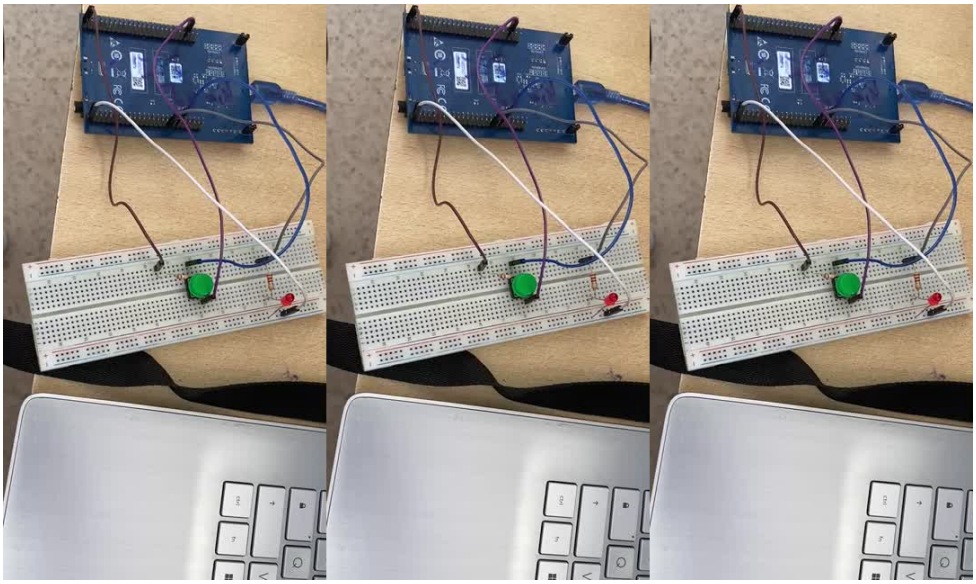


Figure 3 — Extraits de la vidéo de démonstration (t ≈ 0 s, 2 s et 4 s) : sans appui, puis appui sur le bouton vert avec LED rouge allumée.

4. Conclusion et bilan

Objectifs validés :

- Prise en main de la carte STM32F407G-DISC1 et de son ST-LINK intégré (alimentation, programmation, exécution)
- Configuration des GPIO : activation d'horloge, sortie push-pull, entrée avec résistance de rappel
- Câblage correct d'une LED avec résistance de limitation de courant
- Mise en œuvre de la temporisation (HAL_Delay) et de la lecture d'entrée avec anti-rebond

Points clés à retenir : l'horloge du port doit être activée avant `HAL_GPIO_Init` ; toute entrée numérique doit avoir un niveau défini (pull-up ou pull-down) ; une résistance en série avec la LED est toujours nécessaire.

Difficultés fréquentes et solutions :

Difficulté	Cause probable	Solution
LED toujours éteinte	LED à l'envers ou horloge du port non activée	Vérifier la polarité (patte longue = anode) et `__HAL_RCC_GPIOx_CLK_ENABLE()`
Bouton déclenchant plusieurs fois	Rebonds mécaniques	Anti-rebond logiciel (10–20 ms) ou filtrage matériel
État du bouton instable	Entrée flottante	Ajouter une résistance de rappel (externe ou interne)
Téléversement impossible	Câble USB charge seule ou driver ST-LINK absent	Utiliser un câble de données, installer le driver ST-LINK

Perspectives : interruptions externes (EXTI) pour le bouton plutôt que la scrutation, PWM pour moduler la luminosité de la LED, puis timers matériels à la place de HAL_Delay.